Partial Failures

some parts fail while the rest keeps running

Watch the 4-min video

In one line: in a distributed system some components fail while others keep running, and a caller often can't tell a lost request from a slow or crashed service, so bound every call with a timeout, retry safely, and isolate failures so one bad dependency can't take everything down.

1 Not all or nothing anymore

On one machine, things usually work or crash completely. In a distributed system, **some parts fail while others keep running**. A request may succeed in three services and fail in the fourth. A database may accept a write and then lose the connection before replying. This is a **partial failure**, and it's the normal state of a large system, not a rare event: with thousands of machines, something is always broken somewhere.

2 The core problem: a timeout tells you very little

When a call doesn't answer in time, the caller can't know what happened. The request may have been lost, the service may have crashed mid-work, it may just be slow, or it may have **done the work** and lost the reply. The last case is the dangerous one: if you retry, you may charge a card twice. That's why retries need **idempotency** (lesson 20).

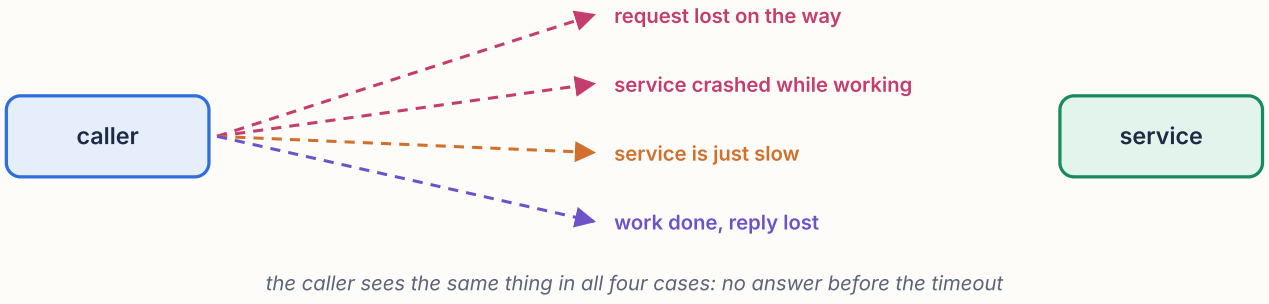


Figure 1. Four different failures, one symptom. The caller only knows that no reply arrived before its deadline.

3 Slow is worse than dead

A dead service fails fast: connections are refused and callers move on. A **slow** service holds every caller's thread, connection and memory while it thinks. Those resources run out, new requests queue up, and the caller fails too. This chain is a **cascading failure**.

WORKED EXAMPLE: A 30-SECOND DEPENDENCY WITH NO TIMEOUT

A downstream service doesn't fail; it starts responding in 30 seconds instead of 50 ms. Your API calls it with no timeout, and each waiting request holds one of 200 worker threads. At 100 requests per second, all 200 threads are stuck within two seconds. Now even endpoints that never touch that dependency can't get a thread, and your whole API is down. Fix: a 1-second timeout on the call, a circuit breaker that opens after repeated failures, a separate thread pool (bulkhead) for that dependency, and a fallback response.

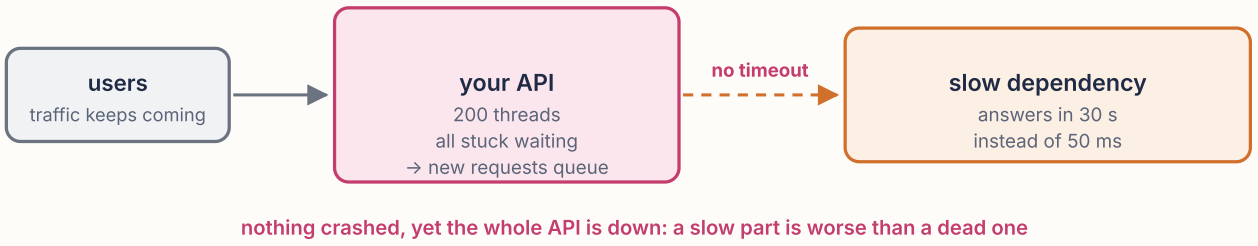


Figure 2. A slow dependency without a timeout ties up every thread, and the failure spreads upstream.

4 The defences

- **Timeouts** on every network call, set from the dependency's real p99, not the default (often infinite).
- **Retries** only for safe (idempotent) operations, with exponential backoff and jitter, and a small retry budget so retries don't double the load.
- **Circuit breakers**: after many failures, stop calling for a while and fail fast, then test with a few requests.
- **Bulkheads**: separate connection or thread pools per dependency, so one can't starve the others.
- **Fallbacks** and graceful degradation: serve cached data, a default, or hide a non-critical feature.

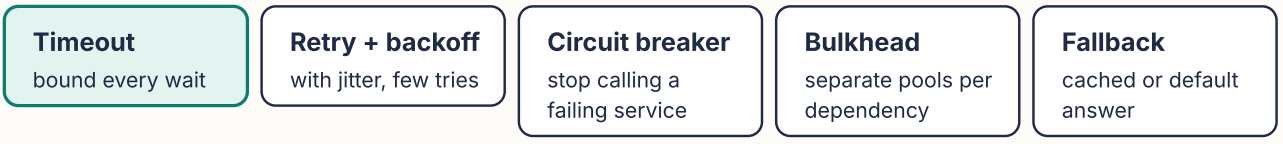


Figure 3. The standard toolkit against partial failure. A timeout comes first: without it, none of the others can act.

5 Design for it

Assume every dependency will be slow, down or wrong at some point. Decide in advance what each feature does then: block, degrade or skip. Test it on purpose with fault injection (chaos testing), because the failure paths are the code paths that never run in a demo.

COMMON MISTAKES

- No timeout, or a timeout longer than the caller's own deadline.
- Retrying non-idempotent calls, like a payment, without an idempotency key.
- Retrying at every layer: 3 retries × 3 layers = 27 calls per user request.
- One shared connection pool for all dependencies.

6 Key terms

Partial failure: some components fail while others keep running

Cascading failure: one failure spreads to callers as their resources run out

Bulkhead: isolated resource pools per dependency

Timeout: maximum time to wait for a reply

Circuit breaker: stops calls to a failing dependency for a while

Graceful degradation: keep core features working when parts fail

7 Interview questions

Q1. What is a partial failure? BASIC

A failure of some components while others keep working, for example one service or one replica down, or a reply lost on the network. The system as a whole is still running, but some requests fail or hang.

Q2. Why is a timeout ambiguous? BASIC

The request may have been lost, the server may have crashed, may be slow, or may have finished the work and lost the reply. The caller can't tell which, so a retry must be safe to repeat.

Q3. One downstream service gets slow and your whole API goes down. Explain and fix. SCENARIO

Calls without timeouts hold threads until the pool is exhausted, so every endpoint stalls. Add timeouts, a circuit breaker, a bulkhead pool for that dependency, and a fallback.

Q4. How do you choose a timeout value? SCENARIO

From the dependency's latency distribution, a bit above its p99, and always below the caller's own deadline. Then pass the remaining deadline down the chain so inner calls don't outlive the request.

Q5. Retries make an outage worse. How? TRICKY

A struggling service gets extra load from every client retrying at once. Use exponential backoff with jitter, cap attempts, use a retry budget, and stop retrying when the circuit breaker is open.

Q6. Is a dead service or a slow service worse for callers? TRICKY

Usually a slow one: dead services fail fast, while slow ones hold the caller's threads and connections, causing cascading failures.

Next: SD-H-003 · Network Partitions backendroadmap.com